Chapter 7

Logarithms

Exponential functions take an exponent and produce a number. Logarithms reverse that process. If the base is b and the result is y, then $\log_b y$ is the exponent that produces y. In other words, a logarithm is not a new kind of arithmetic so much as a way of asking an exponential question backward.

This week has two main goals. The first is to become fluent moving between exponential and logarithmic language: $b^x = y$ and $\log_b y = x$ say the same fact in two different orders. The second is to see why logarithms are useful. They turn multiplication into addition, let us solve for unknown exponents, and give a natural scale for measuring information.

7.1 The Logarithm as an Inverse

The exponential $f(x) = b^x$ with $b > 0$, $b \neq 1$ is one-to-one: distinct exponents give distinct results. That means the function can be reversed. The inverse takes the output of the exponential and returns the exponent that created it. This inverse function is the logarithm base b.

Definition: Logarithm. For base $b > 0$, $b \neq 1$, the *logarithm* $\log_b y$ is the exponent to which b must be raised to produce y:

$$\log_b y = x \quad \text{exactly when} \quad b^x = y.$$

The two statements $b^x = y$ and $\log_b y = x$ say the same thing in opposite directions. Reading one into the other is the single most useful move in the subject. When a logarithm looks mysterious, translate it back into an exponential sentence and ask what exponent is being named.

Key idea. $\log_b y$ is an exponent: the exponent that turns b into y. The exponential and logarithm undo each other,

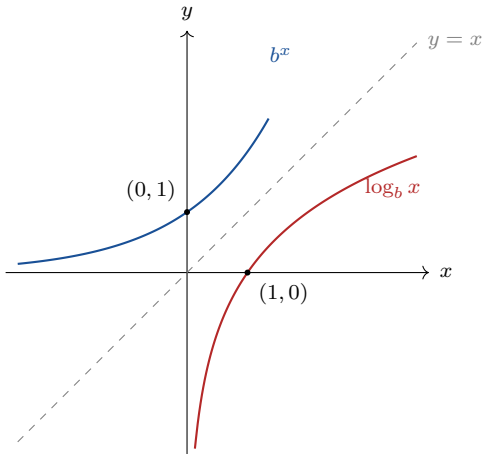
$$b^{\log_b y} = y \qquad \text{and} \qquad \log_b(b^x) = x,$$

because each reverses what the other did.

Example 7.1.1. *(evaluating by asking for the exponent)* $\log_2 8 = 3$, because $2^3 = 8$. $\log_{10} 1000 = 3$, because $10^3 = 1000$. $\log_5 1 = 0$, because $5^0 = 1$. $\log_3 \frac{1}{9} = -2$, because $3^{-2} = \frac{1}{9}$. Each value is found by asking what exponent on the base yields the argument.

The example is deliberately simple. At this stage, evaluating a logarithm is not about a calculator button. It is about hearing the question correctly: "what exponent on this base gives this number?" Once that question is clear, the calculator only helps when the exponent is not one we can recognize mentally.

Because the logarithm is the inverse of the exponential, its graph is the reflection of the exponential graph across the line $y = x$, with domain and range swapped. The exponential has domain all reals and range $y > 0$; the logarithm therefore has domain $x > 0$ and range all reals. Logarithms of zero and of negatives do not exist, since no exponent on a positive base produces them.



The exponential passes through $(0, 1)$; reflected, the logarithm passes through $(1, 0)$, recording that $\log_b 1 = 0$ for every base. The y-axis is a vertical asymptote of the logarithm, the reflection of the exponential's horizontal asymptote. The graph climbs forever, but slowly. That slow climb is one reason logarithmic scales are useful for quantities that range over many orders of magnitude.

Example 7.1.2. *(two named bases)* Base 10 gives the *common logarithm*, written $\log x$. Base e gives the *natural logarithm*, written $\ln x$, the inverse of e^x promised in the previous chapter: $\ln b$ is the exponent that turns e into b. So $\ln e = 1$ and $\ln 1 = 0$.

The base changes the unit in which the exponent is measured. Base 10 measures powers of ten, base 2 measures powers of two, and base e is the natural base that appears in continuous growth. The inverse idea is the same in every base.

Interactive. In the Function Fit Lab, set the parent function to a logarithm and tune $A\,f(b(x-h))+k$ to match a noisy point cloud. The logarithmic shape rises quickly near its vertical asymptote and then flattens, the mirror of the exponential's slow start and rapid climb; fitting one is reading the inverse relationship in data.

`https://bobovski66.github.io/Precalculus/demos/function_fit.html`

7.2 The Laws of Logarithms

Logarithm rules are exponent rules seen through the inverse. Multiplying powers adds exponents, so a logarithm turns a product into a sum. Dividing powers subtracts exponents, so a logarithm turns a quotient into a difference. Raising a power to a power multiplies exponents, so a logarithm turns an exponent into a coefficient.

This is the point at which logarithms become a working tool. They let us replace a multiplicative expression with an additive one. That does not make the expression smaller in every case, but it often makes the structure easier to read and the algebra easier to solve.

Key idea. Laws of logarithms. For $b>0$, $b\neq 1$, and positive M, N,

$$\log_b(MN)=\log_b M+\log_b N,\quad \log_b\frac{M}{N}=\log_b M-\log_b N,\quad \log_b(M^p)=p\log_b M.$$

The picture. A logarithm converts the scale of multiplication into the scale of addition. Products become sums, quotients become differences, and powers become products. This is why logarithms turn the wide range of multiplicative quantities, populations, intensities, probabilities, into manageable additive scales.

Example 7.2.1. *(expanding and combining)* Expand $\log_b\dfrac{x^3y}{z}$:

$$\log_b\frac{x^3y}{z}=\log_b x^3+\log_b y-\log_b z=3\log_b x+\log_b y-\log_b z.$$

Run backward, $2\log x+\log y$ combines to $\log(x^2y)$, collapsing a sum of logarithms into the logarithm of a product.

The two directions serve different purposes. Expanding separates the factors so their roles are visible. Combining compresses several logarithmic terms into one expression. In both directions, the algebra is controlled by multiplication, division, and powers inside the logarithm.

Common pitfall. $\log_b(M + N)$ has no expansion; the laws apply to products, quotients, and powers, never to sums. In particular $\log_b(M+N)$ is not $\log_b M+\log_b N$. The right-hand side equals $\log_b(MN)$, an entirely different quantity.

7.3 Change of Base

A calculator usually computes logarithms in base 10 and base e, yet many questions come naturally in other bases. Computer memory uses base 2. Repeated doubling uses base 2. Repeated tripling uses base 3. The change-of-base formula lets us express a logarithm in any base using logarithms in a base the calculator already knows.

The formula follows from the inverse relationship. Writing $x = \log_b N$ means $b^x = N$. Taking $\log_c$ of both sides and using the power law gives $x \log_c b = \log_c N$, and then solving for x produces the formula.

Key idea. Change of base. For positive bases $b, c \neq 1$ and positive N,

$$\log_b N = \frac{\log_c N}{\log_c b}.$$

Taking $c = 10$ or $c = e$ lets any logarithm be computed from common or natural logarithms.

Example 7.3.1. *(computing a base-2 logarithm)* Find $\log_2 10$. By change of base,

$$\log_2 10 = \frac{\ln 10}{\ln 2} = \frac{2.302585}{0.693147} \approx 3.3219.$$

So $2^{3.3219} \approx 10$, which checks: 10 sits between $2^3 = 8$ and $2^4 = 16$, closer to 8.

The decimal answer is not meant to hide the meaning. It says that 10 is a little more than three doublings from 1. Change of base gives a numerical value for an exponent that does not land exactly on an integer.

Example 7.3.2. *(solving an exponential equation)* Solve $3^x = 20$. Take the logarithm of both sides and apply the power law:

$$x \log 3 = \log 20, \qquad x = \frac{\log 20}{\log 3} = \frac{1.30103}{0.47712} \approx 2.727.$$

The logarithm extracts the exponent, which is exactly what it was built to do.

This same method solves any equation in which the unknown is trapped in an exponent. First take a logarithm of both sides; then use the power law to bring the unknown down where ordinary algebra can reach it.

7.4 Logarithms and Information

Logarithms also measure information. This is not a separate trick from the earlier rules. Information is counted by the number of possibilities that have been narrowed down, and possibilities multiply. Since logarithms turn multiplication into addition, they give the natural additive scale for information.

Consider a quantity that can take one of N equally likely values, and ask how many yes/no questions are needed to identify which value occurred. Each question, answered optimally, halves the number of remaining possibilities. Starting from N and halving down to 1 takes $\log_2 N$ questions, and that count is the number of *bits* of information the answer carries.

Key idea. The information content of selecting one outcome from N equally likely outcomes is $\log_2 N$ bits, the number of yes/no questions needed to pin it down. Equivalently, an event of probability p carries $\log_2 \dfrac{1}{p} = -\log_2 p$ bits: the rarer the event, the more information its occurrence conveys.

Example 7.4.1. *(bits to identify an outcome)* A fair coin has $N = 2$ outcomes, so a flip carries $\log_2 2 = 1$ bit, a single yes/no answer. A byte chooses one of $N = 256$ states, carrying $\log_2 256 = 8$ bits, since $2^8 = 256$. Identifying one card from a deck of 52 needs $\log_2 52 \approx 5.70$ bits: five questions are not quite enough, six more than suffice.

Example 7.4.2. *(information from probability)* An event of probability $p = \frac{1}{32}$ carries $\log_2 32 = 5$ bits when it occurs, because $-\log_2 \frac{1}{32} = \log_2 32 = 5$. A near-certain event of probability $p = 0.99$ carries $-\log_2 0.99 \approx 0.0145$ bits: its occurrence is almost no surprise, so it conveys almost no information. Rare events are informative; common events are not.

The probability version says the same thing in a different language. A rare event rules out many alternatives when it occurs, so it carries more information. A nearly certain event rules out almost nothing, so its information content is close to zero.

The picture. Information is measured on a logarithmic scale because possibilities multiply while information adds. Two independent yes/no choices give $2 \times 2 = 4$ outcomes but $1 + 1 = 2$ bits; ten such choices give $2^{10} = 1024$ outcomes and 10 bits. The logarithm is the bridge that turns the multiplying count of possibilities into the adding count of bits.

Example 7.4.3. *(combining independent sources)* A system has two independent components, one with 8 states and one with 4. The combined system has $8 \times 4 = 32$ states, carrying $\log_2 32 = 5$ bits. The same total comes from adding the parts: $\log_2 8 + \log_2 4 = 3 + 2 = 5$. The product law of logarithms is precisely the additivity of information across independent sources.

7.5 Exercises

Exercises 7.5

7.5.1. Evaluate by asking for the exponent: (a) $\log_3 81$; (b) $\log_{10} 0.01$; (c) $\log_2 \frac{1}{16}$; (d) $\log_7 1$; (e) $\ln e^4$.

7.5.2. Rewrite each exponential statement as a logarithmic one and conversely: (a) $5^3 = 125$; (b) $\log_2 32 = 5$; (c) $e^0 = 1$.

7.5.3. Sketch $f(x) = 2^x$ and $f^{-1}(x) = \log_2 x$ on one set of axes, mark $(0, 1)$ and $(1, 0)$, and identify the asymptote of each.

7.5.4. Expand fully: (a) $\log_b(x^2 y z^3)$; (b) $\log \dfrac{\sqrt{x}}{y^4}$; (c) $\ln \dfrac{e^2 x}{y}$.

7.5.5. Combine into a single logarithm: (a) $\log x + 3\log y$; (b) $2\ln x - \frac{1}{2}\ln y$; (c) $\log_b 6 - \log_b 2$.

7.5.6. Explain why $\log_b(M + N)$ cannot be expanded by the laws, and state what $\log_b M + \log_b N$ does equal.

7.5.7. Use change of base to compute, to four decimals: (a) $\log_2 50$; (b) $\log_5 100$; (c) $\log_3 7$.

7.5.8. Solve for x, leaving exact and decimal forms: (a) $2^x = 15$; (b) $5^x = 2$; (c) $e^x = 10$.

7.5.9. How many bits of information are carried by identifying one outcome from (a) 16 equally likely outcomes; (b) 1000 equally likely outcomes; (c) a single roll of a fair six-sided die? Use change of base where needed.

7.5.10. An event has probability $p = \frac{1}{64}$. Find the information in bits its occurrence conveys, and explain why a rarer event carries more information than a common one.

7.5.11. A password is one of 2^{40} equally likely strings. State its information content in bits, and explain how the product law makes the bits of independent characters add.

7.5.12. Two independent dice, one fair six-sided and one fair four-sided, are rolled. Find the total number of outcomes, the information in bits, and verify that the bits of the two dice add to the bits of the combined system.